

OMG Quality Assessment Report

OMG Core Platform v1.0 — static code audit across backend, database and frontend, plus live functional testing against the production environment. 2026-08-25.

59/₁₀₀ **CONDITIONAL**

Demo-ready with known caveats.
Not production-ready for untrusted multi-user deployment.

1. Executive Summary

This is an independent quality pass over OMG Core Platform v1.0, run as a feature-freeze quality audit — no new functionality, no architecture changes, no scope expansion. The objective was to surface real defects: correctness bugs, RBAC gaps, data-integrity risks, and usability dead ends.

Three methods were used, and every finding below is labeled by which one produced it:

- **Static code audit** — three independent read-only passes over the full backend controller (785 lines / 85 endpoints), the full Prisma schema (1,346 lines / 44 models), and the frontend routing/navigation layer. Every finding cites a file and line.
- **Live functional testing** — the frontend was run against the actual production backend (`https://orchestrai-omg.onrender.com`) and live Neon database, because no separate staging environment exists. A clearly-tagged test asset (`QA-TEST-Journey1-Asset`) was created, taken through registration and risk assessment, and then deleted via the API afterward — confirmed removed (`DELETE` returned `200` , subsequent `GET` returned `404`).
- **Not executed this pass** — several phases of the original brief (95%+ automated unit coverage, load testing at 100/500/1,000/5,000 assets, the remaining four end-to-end journeys, a full 7-role × 59-route access matrix) were not run. These are listed explicitly in §8 rather than estimated or fabricated. No test runner currently exists in the repo (`npm test` is not wired to a framework), and no synthetic data generator exists for load testing — both are real prerequisites, not just time constraints.

Headline result: the platform's core CRUD and reasoning-engine logic works correctly when used as intended — asset registration and risk classification were verified live, end to end, computing the correct result (Critical Risk from Customer-Facing + PII/Sensitive + Critical decision impact +

Automated oversight). But three systemic gaps run through nearly every finding below: **(1)** there is no request-body validation anywhere in the API, **(2)** the database enforces almost none of the integrity/accountability guarantees the platform's own documentation claims, and **(3)** the frontend does not gate write actions by role — it relies entirely on the backend to reject them, silently, with no user-facing explanation.

Certification scores are in §9. Overall: 59 / 100 — Conditional. Demo-ready with known caveats; not production-ready for untrusted multi-user deployment.

2. Platform Inventory (Phase 1)

CATEGORY	COUNT	SOURCE
Frontend page components	62 files (<code>frontend/src/pages/*.tsx</code>)	Frontend nav audit
Registered routes	59 governed + 3 future-placeholder + <code>/login</code>	<code>frontend/src/App.tsx</code>
Navigation menu entries	59 (1:1 with governed routes)	<code>frontend/src/config/navigation.ts</code>
Orphaned pages (built, unreachable)	1 — <code>DecisionGovernancePage.tsx</code>	Frontend nav audit
Backend REST endpoints	85, in a single 785-line controller	<code>backend/src/app.controller.ts</code>
Database models	44	<code>backend/prisma/schema.prisma</code> (1,346 lines, fully read)
Enums	42	same
Relation fields	46 directly enumerated (~51 incl. back-relations)	same
Secondary indexes (<code>@@index</code>)	0	same — known, previously tracked gap
User roles (actual)	7: <code>SUPER_ADMIN</code> , <code>GOVERNANCE_ADMIN</code> , <code>RISK_OFFICER</code> , <code>BUSINESS_OWNER</code> , <code>VALIDATOR</code> , <code>AUDITOR</code> , <code>VIEWER</code>	<code>schema.prisma</code> <code>UserRole</code> enum
Guided tour steps	36	<code>frontend/src/config/landingContent.ts</code> <code>TOUR_STEPS</code>
Demo personas	7 (one per role)	<code>frontend/src/services/mockData.ts</code> <code>DEMO_PERSONAS</code>

Correction to the original test brief: the brief assumed roles "Admin, Executive, Risk Officer, Compliance Officer, Auditor, Viewer." The platform's actual roles are different: `SUPER_ADMIN`, `GOVERNANCE_ADMIN`, `RISK_OFFICER`, `BUSINESS_OWNER`, `VALIDATOR`, `AUDITOR`, `VIEWER`. There is no "Executive" or "Compliance Officer" role in the implementation — `BUSINESS_OWNER` and `VALIDATOR` exist instead. All RBAC findings below use the real role names.

3. Defect Register

Findings are grouped by area, severity-sorted within each group. Each entry states its **source** (Live-Verified = directly reproduced in the running app against production; Static = confirmed by direct code/schema reading, not executed).

3.1 DATA INTEGRITY (DATABASE)

D-1 CRITICAL Static **AIAsset** **cascade-delete silently destroys the entire governance/audit trail.** `backend/prisma/schema.prisma` — every one of ~20 child models (`RiskAssessment`, `DecisionRecord`, `EvidenceDocument`, `Finding`, `GovernanceIncident`, `KillSwitchRecord`, `OverrideRecord`, `RetirementRecord`, `ChangeRequest` and its approvals/transitions/history, `EvidenceRecord` and its mappings, `GovernanceFinding`, `RecommendedAction`, etc.) declares `onDelete: Cascade` against `AIAsset`. There is no soft-delete flag anywhere in the schema (`deletedAt` / `isDeleted` does not exist on any model). One `DELETE /api/assets/:id` call permanently erases every decision, evidence, finding, incident, kill-switch, and retirement record for that asset — with no DB-level barrier. For a platform whose value proposition is a permanent, defensible audit trail, this is the most severe integrity gap found. *Fix*: change evidence/decision/finding/incident-class children to `Restrict`, forcing an explicit archival workflow in application code rather than a silent cascade.

D-2 CRITICAL Static + Live-Verified **"Named accountability" is not enforced anywhere — all owner fields are nullable, and the app does not fill the gap.** Schema: `AIAsset.accountableOwner/governanceSponsor/humanOverrideAuthority/killSwitchAuthority/reassessmentAuthority` and `AssetOwner.businessOwnerId/technicalOwnerId/riskOwnerId/complianceOwnerId/approverId` are all optional. Live-verified: the "Register New AI Asset" modal labels **Accountable Owner***, **Governance Sponsor***, **Risk Owner***, **Technical Owner*** with an asterisk implying required, but none of the four carry the HTML `required` attribute. A test asset (`QA-TEST-Journey1-Asset`) was registered with **all four owner fields left blank** and it was accepted and persisted to the live Neon database with no error, no warning, and no visual indication anything was missing. The platform's own landing page states "Five named owners on every asset" as a governance guarantee; today, zero is also accepted. *Fix*: make ownership fields required in both the form (real client-side validation, not just a visual asterisk) and the API (reject writes past `DRAFT` status without them); consider a DB check constraint as a second layer since app-layer checks can be bypassed by direct writes or seed scripts.

D-3 HIGH Static **Deleting a user silently erases ownership/approval history.** `AssetOwner`'s five owner FKs to `User` have no `onDelete` specified; since all are optional, Prisma defaults to `SetNull`. Hard-deleting a user wipes every `AssetOwner` record's link to them with zero trace of who it used to be — unlike `AuditLog`, which does snapshot `userName` / `userRole` for exactly this reason.

D-4 HIGH Static **GovernanceFinding.policy cascades, contradicting the model's own documented intent.** The schema comment directly above `GovernanceFinding` states findings are "a persisted, owned record... not recomputed on every read" — i.e., meant to outlive the policy that generated them. The actual relation (`schema.prisma:1139`) is `onDelete: Cascade`, so retiring a policy destroys every finding ever raised under it, including resolved ones that should remain as history.

D-5 HIGH Static `PolicyViolation / PolicyMapping` **cascade from `Policy`, destroying compliance-violation history on policy deletion.** Same pattern as D-4: deleting a `Policy` row wipes its violation history, which should survive policy churn.

D-6 HIGH Static `KillSwitchRecord.status` **is a raw, unconstrained string, not an enum.** Of all status-bearing fields in the schema, the emergency-stop control is one of the few with no enum constraint (`schema.prisma:368`) — any string is accepted, including typos, with no DB-level catch. This is a meaningful gap specifically because it's the platform's safety-critical control.

D-7 HIGH Static **"Append-only audit log" is a policy claim, not a schema-enforced guarantee.** `AuditLog` has no `updatedAt`, but nothing else — no DB permission restriction, no trigger, no hash-chaining field — prevents `auditLog.update()` / `.delete()` from being called exactly like any other model. The guarantee currently rests entirely on application-code discipline.

D-8 MEDIUM Static, cross-confirmed by two independent audits `AIAsset.name` **has no uniqueness constraint.** Both the backend and database audits independently flagged this: the central registry object can have exact duplicate names with no DB objection, and there is no tenant/org-scoping field anywhere in the schema to justify it as intentional multi-tenant behavior.

D-9 MEDIUM Static `AuditLog.userId / assetId` **are optional with implicit `SetNull`, producing orphaned/unreachable audit rows over time.** As users and assets are removed, audit rows accumulate with null FKs — still present but unreachable via the normal object graph, in the one table where discoverability matters most.

D-10 MEDIUM Static **Three overlapping, unreconciled state-machine concepts for the same `AIAsset`** (`status`, `governanceState`, and the separate `StateTransition.fromState/toState` enum) — nothing enforces they stay consistent with each other.

D-11 MEDIUM Static **Required `effectiveDate / reviewDate` with no default forces placeholder data on `DRAFT` records** (`Policy`, `CompliancePack`, `RegulatorySource`) — inconsistent with `ChangeRequest`'s equivalent lifecycle dates, which are correctly optional.

D-12 MEDIUM Static **No uniqueness guard against two simultaneously-`ACTIVE` evidence records of the same type for the same asset** — undermines "which evidence is authoritative," the model's own stated purpose.

D-13 MEDIUM Static `ChangeHistoryEntry.assetId` **is a plain string, not a real foreign key** — a denormalized copy with no DB-level guarantee it matches its parent.

D-14 MEDIUM Static, possible **Several "loose reference" fields are deliberately unenforced pseudo-FKs** (`EvidenceRecord.riskAssessmentRef` and siblings) — by design per the schema's own comments, but worth confirming app-layer validation actually exists, since Postgres has no knowledge of them as relations.

D-15 MEDIUM Static **Status/severity modeled as raw strings instead of enums on several audit-relevant models**, inconsistent with the schema's own enum discipline elsewhere (e.g. `PolicyViolation.severity` is a string while its sibling `PolicyViolation.status` correctly uses an enum, in the same model).

D-16 MEDIUM Static **No tamper-evidence mechanism on `AuditLog`** — no hash-chaining/signature field that would let a forensic review detect rows altered via direct database access.

D-17 MEDIUM Static **Deep cascade chains in the Compliance Pack and Regulatory Knowledge frameworks** (`CompliancePack` → `Requirement` → `Control` → `EvidenceMapping` , `RegulatorySource` → `Requirement` → `Obligation` → `Control` → `EvidenceMapping`) silently destroy mapping/evidence-linkage history when a top-level pack or source is deleted.

D-18–D-21 LOW Static Enum sprawl (four functionally-identical severity enums under different names); `GovernanceFindingStatus` lacks a `DISMISSED` -equivalent value its sibling enum has; `PolicyStatus` enum ordering breaks the `DRAFT` -first convention every sibling enum follows; minor uniqueness gaps on `Obligation.name` / `RegulatorySource.name` / `GovernancePolicy.name` .

3.2 API CORRECTNESS & VALIDATION (BACKEND)

A-1 HIGH Static **Every PATCH/DELETE-by-id endpoint returns an unhandled 500 instead of a clean 404 on a bad or already-deleted id.** This pattern repeats identically across roughly 35 endpoints (20 PATCH + 15 DELETE) — every write-by-id route in the file. Prisma throws `P2025` when the target row doesn't exist; nothing catches it, and there is no global exception filter in `main.ts` , so it surfaces as a generic `500 Internal server error` . The codebase already knows the correct pattern — `getAsset` and `getEvidenceRecord` do `findUnique` + `throw new NotFoundException(...)` — it just was never applied to any write path. *Repro:* `PATCH /api/assets/does-not-exist` with any body → 500 instead of 404. *Fix:* wrap every write-by-id handler in the existing find-then-act pattern, or add a global exception filter that maps Prisma's `P2025` to 404.

A-2 HIGH Static **No request-body validation anywhere — every POST/PATCH forwards `@Body()` body: any straight into Prisma.** No DTO classes exist anywhere in `src/` , and `main.ts` never calls `app.useGlobalPipes(new ValidationPipe(...))` . A malformed or empty body (e.g. `POST /api/assets` with `{}`) throws `PrismaClientValidationError` , which is not an `HttpException` , so it also becomes a generic 500 instead of a clean 400 naming the missing field. True for essentially all 46 create/update endpoints.

A-3 MEDIUM Static **Free-text status/severity columns accept anything and are silently persisted as-is** — directly enabled by A-2; concrete case of "silent garbage persistence."

A-4 MEDIUM Static **Mass assignment lets the caller forge server-managed timestamps** — e.g. `POST /api/evidence-records` with a caller-supplied `createdDate` is accepted verbatim, backdating a governance evidence record's provenance. Distinct from the already-tracked "audit log is client-side" gap — this is about the evidence/decision records themselves.

A-5 MEDIUM Static **Client-supplied primary keys: missing-id and duplicate-id both surface as 500 instead of 400/409.** Eleven "registry" models (`CompliancePack` , `ComplianceRequirement` , `RegulatoryRequirement` , `GovernancePolicy` , etc.) use client-supplied string IDs with no `@default` . A missing id throws a validation error → 500; a duplicate id throws Prisma's unique-constraint error (`P2002`) → 500, instead of a clean 409.

A-6 MEDIUM Static **Foreign-key references aren't pre-validated — correctly rejected by the DB, but surfaced as a raw 500 with no indication which field was bad.** The good news:

Prisma's FK constraint does prevent the orphan row from being written (no relational corruption). The defect is purely error-handling quality.

A-7 LOW-MEDIUM Static, cross-confirmed `AIAsset.name` **has no uniqueness constraint** — same as D-8, found independently by both audits.

A-8 LOW Static **Hardcoded demo-persona names used as silent production fallback values** — `app.controller.ts:724` defaults a missing `owner` to `'David Chen (Governance Admin)'`; `:761` defaults a missing `assignedTo` to `'Sarah Jenkins'`. These read as leftover demo-seed values that will misattribute real governance records once used against production data.

A-9 LOW Static **No pagination on most list endpoints; some 2–3-level nested include s** (`getAssets`, `getRegulatorySources`) — unbounded response payloads as row counts grow, distinct from the already-tracked indexing gap.

(Items explicitly checked and found clean: CORS is locked to an explicit origin allow-list, not wide open; every endpoint except `GET /api/health` carries a role guard — no unintentionally open routes; `AUDITOR` and `VIEWER` are never granted a write role across all ~46 write endpoints, confirming the backend's own role semantics are internally consistent.)

3.3 ROLE-BASED ACCESS CONTROL (LIVE-VERIFIED + STATIC)

R-1 HIGH Live-Verified **Write-action buttons are not role-gated on the Asset Registry page — a Viewer sees fully-enabled controls that silently do nothing.** Switched the running app's role selector to `VIEWER` (confirmed via DOM inspection: `select.value === "VIEWER"`). The "Register New AI Asset" button and every row's "Edit"/"Risk" buttons remained visible and enabled — identical to Super Admin (7 Edit + 7 Risk buttons counted, `disabled: false`). Clicking "Register New AI Asset" as Viewer opened **no modal and produced no error message, toast, or console warning** — the button simply does nothing, indistinguishable from a broken button. The backend does correctly reject `VIEWER` on write endpoints (confirmed by A-audit), but the frontend gives the user zero indication why — this is worse than a permission error, because it looks like the application is malfunctioning. *Fix:* gate action buttons by `hasPermission` /role at render time (hide or disable with a tooltip), and if a write is attempted and rejected server-side, surface the actual error to the user instead of failing silently.

R-2 HIGH Static **Seven Release 6–10 modules are unreachable for every role except SUPER_ADMIN.** `frontend/src/services/mockData.ts` `DEMO_PERSONAS` allow-lists for `GOVERNANCE_ADMIN`, `RISK_OFFICER`, `BUSINESS_OWNER`, `VALIDATOR`, `AUDITOR`, `VIEWER` do not include `/mapping-workspace`, `/requirement-registry`, `/obligation-library`, `/governance-intelligence`, `/governance-actions`, `/decision-traceability`, `/governance-studio` — all seven are registered, routed, and present in the nav config, but `AuthContext.hasPermission` checks the persona's `allowedNav` array and blocks them for every role except Super Admin (which bypasses the list entirely). These represent the newest and most heavily-featured releases (6 through 10) of the platform. Signing in as any role other than Super Admin hides these sidebar entries entirely, and direct URL access is blocked by `ProtectedRoute`. This looks like the allow-lists were simply never updated when these modules shipped.

R-3 MEDIUM Static, possible **Auditor persona has /rbac access; Governance Admin does not.** The Auditor persona is documented as "Independent Auditor • Read-only," yet is one of only two

personas (with Super Admin) granted the RBAC Administration screen — while Governance Admin, a broader operational role, is not. Not verified whether the page itself enforces read-only rendering for non-Super-Admin viewers.

3.4 USABILITY & NEGATIVE-PATH HANDLING (LIVE-VERIFIED)

U-1 MEDIUM Live-Verified **The Risk Assessment Wizard can be clicked through all 6 steps with zero selections at any step, landing on a completely blank Summary panel with no submit button and no explanation.** Reproduced twice for certainty. Clicking "Next Step →" six times in a row with no option selected at any of the five input steps advances the wizard fully to "6. Summary," which renders no risk tier, no field summary, and — critically — **no submit/apply button at all**, only "← Previous Step." A user in this state has no way to know they must go back and select something; the wizard gives no validation message at any step blocking progression. Retested with real selections at every step (Customer Facing / PII-Sensitive / Critical / Critical Operational / Automated) — the Summary correctly rendered "Critical Risk" and an "Apply & Update Asset Risk Profile" button, which worked correctly and persisted the change (`Current Risk: Critical Risk` updated live on the asset list, verified via the API-backed persistence log). *Fix:* either block advancing past a step with no selection, or render an explicit "select an option for each step" message on the Summary panel when data is missing, rather than an empty panel.

U-2 LOW Static **Icon-only buttons in the top navigation and asset-registry toolbar have no accessible name** (`aria-label` absent on several `<button>` elements identified during live DOM inspection — e.g. the sidebar collapse toggle, several icon-only action buttons). A screen-reader user cannot determine their function from the accessibility tree alone.

3.5 NAVIGATION & REGRESSION (STATIC)

N-1 MEDIUM Static **`DecisionGovernancePage.tsx` is a fully-built, completely orphaned page** — an 8-point decision-readiness checklist wired to real service calls, but not imported in `App.tsx` and not in `navigation.ts`. Dead code, unreachable through any UI path. Likely superseded by `DecisionWorkbenchPageV4` (which is what's actually routed at `/decision-workbench-v4`) and never deleted.

N-2 LOW Static **Stale step-count in a code comment in `GuidedTour.tsx`** — comment says "currently 35 steps"; the actual array has 36. No user-facing effect since the component reads the array length dynamically, but signals the array was edited without updating the comment.

N-3 LOW Static **Domain-grouping comments disagree between `App.tsx` and `navigation.ts`** for `/decision-traceability` and `/governance-studio` — the actual route paths match correctly (no functional defect), but the two files' organizational comments would mislead a future maintainer.

(Checked and found clean: all 59 nav entries have a matching registered route and vice versa — no broken nav links, no unreachable orphan pages other than N-1; all spot-checked guided-tour step routes resolve to real pages; landing page numeric claims — 9 domains, 56 modules, 85 endpoints, 44 models — were cross-checked against the inventory in §2 and are accurate.)

4. End-to-End Journey Results

Journey 1 — Register Asset → Risk Assess → [Approve → Add Evidence → Review → Trace Decision]: executed live through Risk Assessment (registration and risk classification both verified correct and persisted). Approval, evidence attachment, review, and decision-trace steps were **not executed** this pass — see §8.

Journeys 2–5 (Compliance Pack creation, Governance Finding → Action → Decision, Rule modification → Intelligence Engine recompute, Decision Evidence Pack generation): **not executed** this pass — see §8.

5. Negative Testing Results

TEST	RESULT
Register asset with all four owner fields blank	Accepted and persisted — see D-2
Skip all 6 steps of the Risk Assessment Wizard	Reaches a dead-end blank Summary with no submit path — see U-1
Attempt a write action (Register/Edit) as VIEWER	UI does not block the click; silently produces no effect , no error shown — see R-1
Bad/deleted id on a write endpoint (static code path, not live-triggered against prod)	Confirmed by code reading to throw an unhandled 500 instead of 404 — see A-1
Duplicate asset name	Confirmed by schema reading to have no uniqueness constraint — see D-8/A-7

6. Scorecard Summary Table (see §9 for full certification)

DIMENSION	SCORE
Architecture	82 / 100
Functionality	78 / 100
Usability	68 / 100
Performance	Not Assessed — see §8
Security	41 / 100
Data Integrity	45 / 100
Demo Readiness	58 / 100

DIMENSION	SCORE
Production Readiness	40 / 100
Overall	59 / 100 — Conditional

7. Root Cause Themes

Nearly all 30+ findings above trace back to three systemic patterns, not 30 unrelated bugs:

1. **No validation layer exists between the HTTP boundary and Prisma.** This single gap (A-2) is the direct or contributing cause of A-1, A-3, A-4, A-5, A-6, and D-2's live reproduction. Fixing it once — a global `ValidationPipe` plus DTOs for the ~46 write endpoints — would close roughly a third of the register in one pass.
2. **The schema optimizes for "never lose a write" over "never lose history."** Cascade is the default relationship behavior throughout, including on the exact models (findings, violations, evidence mappings) whose entire purpose is to outlive the parent record's lifecycle. This is a one-line-per-relation fix (`Cascade` → `Restrict` or `SetNull` with a denormalized snapshot) but touches ~15 relations.
3. **The frontend trusts the backend to enforce permissions and shows nothing when it does.** R-1 and R-2 are two faces of the same gap: role information exists and is even checked in places (`hasPermission`), but the UI doesn't consistently act on it, and when the backend correctly rejects an action, nothing tells the user why.

8. Explicitly Not Covered This Pass

Stated plainly rather than estimated, per the platform's own "no hallucination" documentation standard:

- **Phase 2 — 95%+ automated unit test coverage.** No test runner is currently wired into either `package.json` (confirmed: no Jest/Vitest config, no existing `*.test.ts` files found during this audit). Building a suite to 95% coverage of the 9 reasoning engines is a substantial standalone engineering effort — recommend scoping as its own follow-on task, starting with the reasoning/computation engines (`governanceReasoningEngine.ts` , `governanceActionsEngine.ts` , `decisionTraceabilityEngine.ts`) before CRUD glue code.
- **Phase 3 — full endpoint-by-endpoint live API testing** (success/invalid/missing/unauthorized/deleted/concurrent for all 85 endpoints). The backend static audit covers correctness of the code paths in detail; live confirmation of every endpoint's exact HTTP response was not performed except where noted above.

- **Phase 4 — live database rollback testing.** Cascade/integrity behavior was verified by reading the schema, not by executing destructive operations against production data.
- **Phase 5 — full 7-role × 59-route access matrix.** One role (`VIEWER`) was live-tested against one page (Asset Registry); the remaining 6 roles × 58 pages were not individually walked.
- **Journeys 2–5** (Compliance Pack, Governance Finding/Action, Rule-change recompute, Decision Evidence Pack) were not executed.
- **Phase 9 — performance testing at 100/500/1,000/5,000 assets.** No synthetic data seeding script exists; the live database currently holds single-digit record counts per model. Given zero secondary indexes exist (a known, already-tracked gap — see D-list note), there is a real, specific performance risk at scale on any filtered or joined query, but no benchmark number is reported because none was measured — reporting one would be fabrication.
- **Phase 10 — full demo certification walkthrough** of every module. Landing page, guided tour entry points, and Journey 1 through risk assessment were verified; the remaining modules were covered only by the static navigation audit (§3.5).

9. Certification Scorecard

Architecture — 82/100. The repository-pattern, cache-then-network, layered design works as documented and was proven live (asset creation and risk computation both round-tripped through the real API to Neon correctly). Deductions: the validation-layer gap (A-2) and the cascade-delete design (D-1) are architectural choices, not implementation slips, and both need a structural fix rather than a patch.

Functionality — 78/100. Core flows work correctly when used as intended — risk classification computed the right tier from the right inputs, live, on the second clean test. Deductions: the wizard dead-end (U-1), the unenforced ownership requirement undermining a platform's stated core feature (D-2), and the already-documented ~18/44 models with no write API.

Usability — 68/100. Landing page and asset registry are well-designed and functional. Deductions: silent no-op buttons for restricted roles (R-1), the wizard dead-end (U-1), missing accessible names on icon buttons (U-2).

Performance — Not Assessed. See §8. Zero database indexes across 51 relations is a documented, known architectural risk factor for behavior at scale, but no number is reported without a measurement.

Security — 41/100. Consistent with the platform's own previously-published Security Review (38%). No authentication (already tracked), no request validation (A-2), inconsistent/raw error responses that can leak implementation detail (A-1, A-6). CORS is correctly locked down, and RBAC role semantics are internally consistent on the backend — real positives, not enough to offset the rest.

Data Integrity — 45/100. The most consequential findings in this report are here: cascade-delete of the entire audit trail (D-1), unenforced accountability (D-2, live-reproduced), and an audit log whose append-only guarantee is convention rather than schema-enforced (D-7).

Demo Readiness — 58/100. The landing page, guided tour, and core asset/risk flow demo well. Real risk: presenting as any role other than Super Admin hides the newest platform capabilities entirely (R-2), and any accidental click on a restricted action produces silence rather than a graceful message (R-1) — both are the kind of thing that reads as "broken" in front of a customer, not "access-controlled."

Production Readiness — 40/100. Consistent with, and slightly below, the existing Production Readiness Assessment's "Fair" rating — the new data-integrity findings in this report (D-1 through D-7) compound the already-known gaps (no auth, no tests, no indexes) rather than introducing new categories of concern.

Overall — 59/100 — Conditional. The platform is genuinely demo-ready for a guided, Super-Admin-driven walkthrough of the core lifecycle, which is what it does today. It is not ready for unsupervised multi-role demos or any production use with untrusted users until the Po items below are addressed.

10. Recommended Remediation Sequence

Po — before any further customer-facing demo: 1. Fix R-1 (gate write buttons by role, surface rejected-write errors) — highest demo-embarrassment risk, smallest fix. 2. Fix R-2 (extend Release 6–10 modules to every persona's `allowedNav`) — one file, `mockData.ts`, restores visibility of the newest work to every role. 3. Fix U-1 (wizard dead-end) — block progression without a selection, or render a clear message on Summary.

P1 — before production hardening begins: 4. Add a global `ValidationPipe` + DTOs (closes A-1 through A-6 largely at once). 5. Enforce ownership fields as required, client- and server-side (D-2). 6. Change cascade behavior on evidence/decision/finding/incident-class relations to `Restrict` (D-1, D-4, D-5, D-17).

P2 — integrity hardening: 7. Convert remaining free-text status/severity fields to enums (D-6, A-3, D-15). 8. Add uniqueness constraints (D-8/A-7, D-12, D-21). 9. Add a hash-chain or equivalent tamper-evidence field to `AuditLog` (D-16). 10. Delete or re-wire `DecisionGovernancePage.tsx` (N-1).

P3 — follow-on scoping, not fixes: 11. Scope and build the Phase 2 unit test suite (reasoning engines first). 12. Build a synthetic data seeder and run the Phase 9 performance benchmarks. 13. Complete the remaining four E2E journeys and the full RBAC access matrix.